

A branch-and-cut approach to the repetition-free longest common subsequence problem

Carlos E. Ferreira¹ and Christian Tjandraatmadja²

*Departamento de Ciência da Computação
Instituto de Matemática e Estatística, Universidade de São Paulo
São Paulo, Brazil*

Abstract

We present a branch-and-cut approach to solve the following problem. Given two sequences x and y over a finite alphabet, find a repetition-free longest common subsequence of x and y . Implementation details of an efficient separation routine and several heuristics are described. We can solve medium size instances of the problem to optimality.

Keywords: Longest common subsequence, integer programming, branch-and-cut.

1 Introduction

In the context of genome rearrangement analysis, a useful measure of similarity between two genomes is the length of a longest common subsequence (LCS) between the genomes. However, this measure does not consider gene duplication, an event that is not usually considered as it is not frequent in

¹ Email: cef@ime.usp.br.

² Email: christj@ime.usp.br. Supported by FAPESP (proc. 07/57149-5).

simple genomes and makes the problem harder. To take this event into account, Sankoff [6] proposed the exemplar model, which consists in searching, for each family of duplicated genes, an exemplar representative of that family.

In this paper, we consider a similarity measure proposed by Adi et al. [1] that takes into account the concept of exemplar genes. It is the length of a longest common subsequence with the restriction that there is at most one representative of each family of duplicated genes in the sequence. Variants of this problem are considered in [2] and [3].

We denote the length of a sequence s by $|s|$, the i -th symbol of s by s_i and the sequence $s_i s_{i+1} \dots s_j$ by $s[i..j]$. A *subsequence* of s is a sequence obtained by removing zero or more symbols from s . A *common subsequence* of sequences s and t is a subsequence of both s and t . A *repetition-free common subsequence* is a common subsequence in which every symbol appears at most once. A *repetition-free longest common subsequence*, or RFLCS, is a repetition-free common subsequence of maximum length. For example, cad is an RFLCS of $caadbd$ and $abacad$.

We are interested in solving the RFLCS problem, which consists in finding a repetition-free longest common subsequence of two finite sequences x and y over an alphabet Σ (assumed implicit throughout the paper). Adi et al. [1] proved that the RFLCS problem is APX-hard even when restricted to sequences in which the number of occurrences of every symbol in both sequences x and y is bounded by two (this also implies that the problem is NP-hard).

A preliminary version of this approach is presented in [4]. In this paper, we focus on some implementation techniques and heuristics that improve the performance of this branch-and-cut approach.

This paper is organized as follows. In Section 2, we present an integer linear program formulation of the problem. The next sections are devoted to details of this branch-and-cut approach: separation, variable elimination, and primal heuristics.

2 IP formulation

An ordered pair (i, j) is a *match* over x and y (of symbol σ) if $x_i = y_j (= \sigma)$. Let M be the set of all matches over x and y .

We say that two matches (i, j) and (k, l) *cross* (or are *crossing*) if $i \leq k$ and $j \geq l$, or $i \geq k$ and $j \leq l$. As visually represented in Figure 1, they cross if the segments representing them intersect each other graphically, including the case in which they have an extreme point in common.

Let A be a set of pairwise non-crossing matches. It is possible to order the matches of A increasingly according to their elements and take their symbols in that order to obtain a sequence s . The sequence s is a common subsequence of x and y and we say that A defines s . For example, if $x = caadb d$ and $y = abacd ad$, the set of matches $\{(1, 4), (2, 6), (6, 7)\}$ defines cad . Furthermore, note that any common subsequence can be constructed in this manner by some set of pairwise non-crossing matches.

We say that two matches (i, j) and (k, l) are *conflicting* if they cross or are of the same symbol. In the same manner as in the previous paragraph, a set of pairwise non-conflicting matches defines a repetition-free common subsequence. Therefore, the RFLCS problem can be translated into finding a set of pairwise non-conflicting matches of maximum cardinality.

An *extended star* is a set of pairwise conflicting matches.

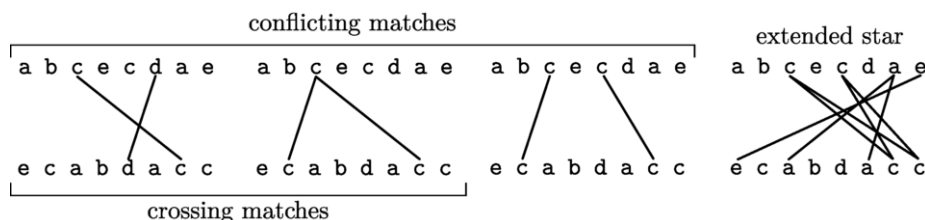


Fig. 1. Examples of crossing matches, conflicting matches, and extended star.

The following integer linear program models the RFLCS problem, since each of its feasible solutions corresponds to a set of pairwise non-conflicting matches of M and vice versa. Let z_{ij} be a binary variable for a match (i, j) in M .

$$\begin{aligned}
 & \max \quad \sum_{(i,j) \in M} z_{ij} \\
 & \text{s. t.} \quad \sum_{(i,j) \in K} z_{ij} \leq 1 \quad \text{for every maximal extended star } K \subseteq M, \\
 & \quad \quad z_{ij} \in \{0, 1\} \quad \text{for every } (i, j) \in M.
 \end{aligned}$$

It has been proved in [4] that the (only) class of inequalities of the formulation, called *extended star inequalities*, is facet-defining.

The goal of this paper is to provide an efficient branch-and-cut procedure to solve the RFLCS problem using the above formulation. In the next section, we describe a polynomial-time algorithm to solve the separation problem for these inequalities.

3 Separation algorithm

The separation problem for extended star inequalities can be defined as follows. Given two sequences x and y over a finite alphabet Σ and a feasible solution z of the linear relaxation of the integer linear program in the previous section, find a maximal extended star K such that $\sum_{(i,j) \in K} z_{ij} > 1$, or inform if there is no such K . Therefore, it suffices to find a maximal extended star of maximum weight using, for every match (i, j) , the value $0 \leq z_{ij} \leq 1$ as the weight of (i, j) (*maximum extended star*).

If we revert one of the sequences, say x , an original match (i, j) turns into a match $(|x| + 1 - i, j)$. Thus, every extended star turns into a set of matches in which any two matches with different symbols are not conflicting. We call this set an *extended planar matching*. Note that finding a maximum extended star in x and y is equivalent to finding a maximum extended planar matching in x^r and y (where x^r is the reverse of x), which is more convenient to handle.

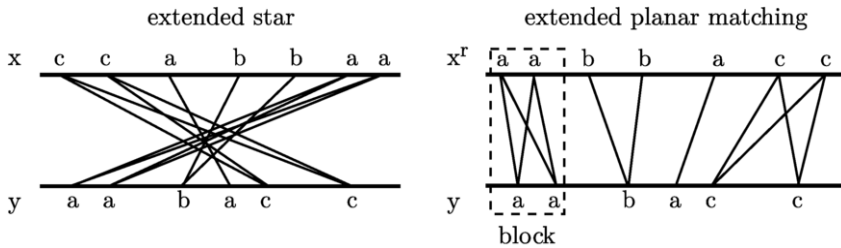


Fig. 2. An extended star and the corresponding extended planar matching.

Define a *block* B as a set of matches of a same symbol σ that is formed by a start match (i', j') , an end match (i, j) with $i \geq i'$ and $j \geq j'$, and every match (k, l) of symbol σ such that $i' \leq k \leq i$ and $j' \leq l \leq j$ (see Figure 2). Two blocks B and C cross if there is a pair of matches $m_B \in B$ and $m_C \in C$ that cross. We can interpret an extended planar matching as a set of pairwise non-crossing blocks.

Let $L_e(i, j)$ be the weight of a maximum extended planar matching of $x[1..i]$ and $y[1..j]$. We adapt the dynamic programming approach for the LCS problem from [7] to find a maximum extended planar matching. The following recurrence for $L_e(i, j)$ is based on this approach and considers weights and blocks in place of matches.

Let $w_b((i', j'), (i, j))$ be the weight of a block that has (i', j') as a start match and (i, j) as an end match (that is, the sum of the weights of the matches of that block). Let $S(i, j)$ be the set of all start matches of blocks that have (i, j) as an end match.

$$L_e(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \max\{L_e(i, j-1), L_e(i-1, j), B_{\max}(i, j)\} & \text{if } x_i^r = y_j, \\ \max\{L_e(i, j-1), L_e(i-1, j)\} & \text{if } x_i^r \neq y_j; \end{cases}$$

$$\text{where } B_{\max}(i, j) = \max_{(i', j') \in S(i, j)} \{L_e(i'-1, j'-1) + w_b((i', j'), (i, j))\}.$$

Observe that $B_{\max}(i, j)$ is the weight of a maximum extended planar matching of $x[1..i]$ and $y[1..j]$ that contains the match (i, j) . It can be computed in polynomial time in a straightforward manner: enumerate every block with start match in $S(i, j)$, calculate the sum above for each block and take the maximum.

The solution $L_e(|x|, |y|)$ can be computed using dynamic programming. It is easy to see that it takes $O(|x|^2|y|^2)$ time. Finding an actual extended planar matching of such weight can be done by storing references to blocks used in the calculation.

Finally, we note that it is possible (although not trivial) to adapt the idea of the Hunt-Szymanski algorithm for the LCS problem [5] to solve the separation problem using the same recursion above, in order to obtain a version of this algorithm that is more efficient in practice.

4 Variable elimination

Given a match (i, j) , denote by $u_{\text{pref}}(i, j)$ and $u_{\text{suff}}(i, j)$ an upper bound for the length of an RFLCS of $x[1..i-1]$ and $y[1..j-1]$, and of $x[i+1..|x|]$ and $y[j+1..|y|]$, respectively.

We say that the function $u(i, j) = u_{\text{pref}}(i, j) + u_{\text{suff}}(i, j) + 1$ is a *match upper bound* for (i, j) .

Lemma 4.1 *Let (i, j) be a match and let $u(i, j)$ be a match upper bound for (i, j) . If there is a repetition-free common subsequence of x and y of length greater than $u(i, j)$, then (i, j) is not in a set that defines an RFLCS of x and y .*

Proof. Let (i, j) be a match and A be a set of matches that defines a repetition-free common subsequence s . Suppose that $(i, j) \in A$. It is easy to show that $u(i, j)$ is an upper bound for the cardinality of A , as follows. Since A contains (i, j) , any other match in A must come from the sequences $x[1..i-1]$ and $y[1..j-1]$, or from $x[i+1..|x|]$ and $y[j+1..|y|]$. Therefore, A can contain at most $u_{\text{pref}}(i, j) + u_{\text{suff}}(i, j) + 1 = u(i, j)$ matches.

Since $|A| \leq u(i, j)$ for every A that contains (i, j) , if there is a repetition-free common subsequence with length greater than $u(i, j)$, then (i, j) is not in a set that defines it, including the case in which it is an RFLCS. $\square$

The lemma above can be used to fix variables to zero as we find better primal solutions to the problem. To do that, we have to compute $u_{pref}(i, j)$ and $u_{suff}(i, j)$. We use as upper bound the length of an LCS or the number of different symbols that occur in both sequences, whichever is lower. Most algorithms that compute the LCS of two sequences [5,7] can be easily adapted to compute the LCS of all prefixes or all suffixes of those sequences without worsening their complexity. Counting the number of different symbols is also straightforward, so, using these upper bounds, $u(i, j)$ can be easily computed.

5 Primal heuristics

In the branch-and-cut procedure, heuristics can be used to obtain better primal bounds, which helps both in obtaining an optimal solution earlier and in fixing variables using the method described in the previous section. The following heuristics use a feasible solution of the relaxation as a parameter. The closer this parameter is to an optimal solution, usually the better the result of the heuristic is. Therefore, these heuristics can be executed several times during the branch-and-cut procedure in attempt to obtain increasingly better integer feasible solutions.

Let z be a feasible solution of the relaxation. The heuristics are as follows.

Weighted LCS and remove repetitions. First, compute a set of pairwise non-crossing matches of maximum weight using z as weight, and then, from the sequence defined by this set, remove repeated occurrences of symbols.

Simple greedy. Greedily choose matches (i, j) with z_{ij} closest to 1, ignoring those that conflict with some match already chosen.

Match upper bound greedy. Greedily choose matches (i, j) with $z_{ij}u(i, j)$ closest to 1, where $u(i, j)$ is the match upper bound of (i, j) , ignoring those that conflict with some match already chosen.³

Heuristics that do not use z can be found in [1]. One can also use the concept of match upper bound to develop a divide-and-conquer heuristic that does not use a feasible solution of the relaxation. The idea is to choose the match with the highest match upper bound, subdivide the sequences into two

³ Intuitively, if a match upper bound is tight, then it gives us a notion of the length of a longest repetition-free common subsequence that contains the match.

(before and after the match) and solve the subproblems recursively. This idea shows good results in the computational tests we performed.

6 Experimental results

The tests were performed in a 2.83Ghz machine running Linux with 32GB of RAM. The inputs x and y are uniformly random sequences of length 1024. The results are the mean of 10 executions with different inputs, each within the time limit of 30 minutes. The implementation is in C and uses the linear programming package *glpk* (version 4.40) for solving linear programs and managing the branch-and-cut tree. Only the extended star cuts described here are used. Some simple preprocessing is done to eliminate unnecessary symbols.

$ \Sigma $	Solution	Running time (s)				VE + PH		
		VE+PH	PH	No VE/PH	DP sep.	Tree	Cuts	Root
1024	56.9	1.699	5.025	5.495	10.307	1.0	140.0	56.9
896	59.2	2.493	8.454	9.782	17.300	1.0	172.0	59.2
768	64.1	4.724	15.039	19.072	28.105	1.0	251.9	64.1
640	70.2	6.789	25.775	30.851	53.552	1.0	306.0	70.2
512	78.1	14.375	63.984	78.752	106.518	1.4	470.0	78.3
384	89.5	100.673	406.354	506.547	533.532	5.6	1145.1	90.2
256	95.7–105.6	1793.710	1800.000	1800.000	1800.000	≥ 3.9	≥ 3075.5	≤ 107.3
128	95.3–127.7	1800.000	1800.000	1800.000	1800.000	≥ 1.0	≥ 235.9	≤ 127.7

In the table above, VE stands for using the variable elimination technique (Section 4), PH stands for using the primal heuristics (Section 5), and DP sep. stands for using the dynamic programming version of the separation algorithm (instead of the Hunt-Szymanski-based version), without VE or PH, which is the same as in [4] aside from minor improvements in implementation. The last three columns are, respectively, the size of the branch-and-cut tree, the total number of extended star cuts generated (both with VE and PH), and the objective value of the root relaxation solution (without VE or PH). In the last two rows, the time limit was always hit, except when $|\Sigma| = 256$ with VE and PH, in which it was hit 8 out of 10 times (in the rest, it was never hit).

From the experimental tests, we observe that it is more difficult to solve instances with small alphabets, likely due to the fact that it implies in a high number of repetitions and variables. The results show that using both the variable elimination technique and primal heuristics improve significantly the running time of the algorithm (remember that the variable elimination

technique depends on good primal heuristics to work well). Also, for alphabets that are not too small, the work done at the root node results in a solution close to the integer optimal solution, which suggests that the extended star cuts are effective in these cases.

For purposes of comparison, we also tested using a weaker class of inequalities: $z_{ij} + z_{kl} \leq 1$ for every $(i, j), (k, l) \in M$ that cross. The program was not able to solve the problem to optimality within 30 minutes with sequences of length 1024 even for large alphabets, using cuts generated by *glpk*.

7 Conclusion

In this paper, we further study the RFLCS problem from a more practical perspective. We present a simple technique for removing variables we know that cannot be in an optimal solution and describe primal heuristics to obtain good feasible solutions during the branch-and-cut process, both of which help to find an optimal solution more quickly. The computational experiments show that the proposed approach is useful to solve instances of medium size.

References

- [1] Adi, S. S., M. Braga, C. G. Fernandes, C. E. Ferreira, F. V. Martinez, M.-F. Sagot, M. A. Stefanos, C. Tjandraatmadja and Y. Wakabayashi, *Repetition-free longest common subsequence*, Discrete Applied Mathematics (2009), to appear.
- [2] Bonizzoni, P., G. D. Vedova, R. Dondi, G. Fertin, R. Rizzi and S. Vialette, *Exemplar longest common subsequence*, IEEE/ACM Trans. Comput. Biol. Bioinformatics **4** (2007), pp. 535–543.
- [3] Bonizzoni, P., G. D. Vedova, R. Dondi and Y. Pirola, *Variants of constrained longest common subsequence* (2009), arXiv/0912.0368.
- [4] Fernandes, C. G., C. E. Ferreira, C. Tjandraatmadja and Y. Wakabayashi, *A polyhedral investigation of the LCS problem and a repetition-free variant*, in: *Proc. of the 8th Latin American Symposium on Theoretical Informatics (LATIN'08)*, Lecture Notes in Computer Science **4957** (2008), pp. 329–338.
- [5] Hunt, J. W. and T. G. Szymanski, *A fast algorithm for computing longest common subsequences*, Communications of the ACM **20** (1977), pp. 350–353.
- [6] Sankoff, D. and J. B. Kruskal, “Time warps, string edits, and macromolecules: The theory and practice of sequence comparison,” Addison-Wesley, 1983.
- [7] Wagner, R. A. and M. J. Fischer, *The string-to-string correction problem*, J. ACM **21** (1974), pp. 168–173.